Домашнее задание 1. Эволюция систем обработки больших данных

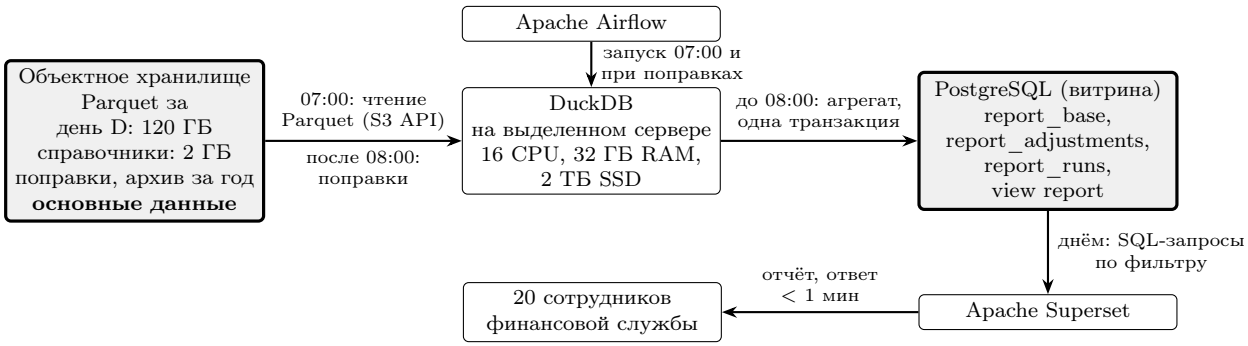
Эль-Сибаи Амир, группа ИИИ-3-ИнжДан-1-2

Кейс 1. Ежедневный отчёт о продажах

1. Требования

- Аналитическая (OLAP) пакетная задача: 120 ГБ Parquet + 2 ГБ справочников, окно с 07:00 до 08:00.
- Результат: агрегат «дата × магазин × категория» с продажами, возвратами и выручкой.
- Днём до 20 человек фильтруют готовый агрегат, ответ нужен быстрее минуты. Исходные строки не нужны, обновлять отчёт в течение дня не надо.
- Исправления за прошлые даты можно публиковать после 08:00. При сбое отчёт может опоздать, но цифры в нём должны быть верными и полными.

2. Схема



3. Обоснование

Как получить и опубликовать отчёт к 08:00. В 07:00 Airflow проверяет, что пришли данные по всем 120 магазинам, и запускает SQL-расчёт в DuckDB. DuckDB читает из хранилища только нужные колонки Parquet, соединяет продажи со справочниками, считает итоги и пишет их во временную таблицу. Дальше проверки: все 120 магазинов на месте, итоги сходятся с суммой по файлам. Если проверки прошли, агрегат одной транзакцией попадает в `report_base` + в `report_runs` появляется запись «дата D, версия N, READY, время публикации». Тяжёлая подготовка данных идёт утром на сервере, лёгкие запросы пользователей днём идут к готовому агрегату в PostgreSQL через Superset. Друг другу они не мешают.

Расчёт. $v_{\min} = \dfrac{120\ \text{ГБ}}{1\ \text{ч}} = \dfrac{120\,000\ \text{МБ}}{3\,600\ \text{с}} \approx 33{,}3$ МБ/с (≈ 267 Мбит/с), со справочниками $\sim 33{,}9$ МБ/с. Это нижняя граница: если весь час уйдёт на чтение, на расчёт и публикацию времени не останется. При 30 минутах на чтение нужно уже $\sim 66{,}7$ МБ/с. Расчёт не учитывает задержки запросов к хранилищу (список файлов, открытие каждого файла), распаковку и разбор Parquet, соединение со справочниками и агрегацию, сброс на SSD при нехватке памяти, проверки, запись и публикацию, повторы при ошибках, чужой трафик в сети. С другой стороны, из Parquet читаются только нужные колонки, так что реально прочитать придётся меньше 120 ГБ. Надо измерить скорость чтения из хранилища на сервер при параллельном чтении + полное время расчёта на данных за реальный день.

Повторный расчёт и исправления. Поправка — строка с разницей со знаком и уникальным `correction_id`. Поправки агрегируются и загружаются в `report_adjustments`, где `correction_id` — первичный ключ (`ON CONFLICT DO NOTHING`). В той же транзакции в `report_runs` появляется версия N+1 со статусом CORRECTED. Итоговый отчёт — представление `report` = `report_base` + сумма поправок. Если тот же файл загрузят ещё раз или задание перезапустится, поправка второй раз не учтётся. Базовый расчёт за D тоже можно спокойно повторить, например после сбоя: он целиком заменяет данные за D в одной транзакции. В Superset пользователь видит только даты со статусом READY или CORRECTED + плашку «Данные за D, версия N, опубликовано в HH:MM». Незавершённый расчёт он не видит.

Кластер не оправдан. На выходе тысячи строк, а данные одного дня колоночный движок обработает на одном сервере. Узкое место — чтение из хранилища, кластер его не расширит, если все узлы читают через тот же канал. Команде из двух SQL-специалистов тяжело обслуживать кластер: сложность растёт, выигрыша нет. К этому вопросу стоит вернуться, только если замеры покажут, что сервер не успевает.

Альтернатива: кластер Hadoop/Spark. Отказался: нужно покупать и обслуживать несколько узлов, настраивать YARN, разбираться с отказами узлов, и всё ради задачи, которую решает один сервер. Spark на одной машине тоже не подходит: сложность та же, масштаба нет.

4. Ограничения

Выбрал простое и недорогое решение + отказался от горизонтального масштабирования и отказоустойчивости. Весь расчёт идёт на одном сервере, если он сломается, отчёт опоздает. Данные при этом не теряются: исходники лежат в хранилище, а тот же SQL-скрипт DuckDB можно запустить на любой временной машине. Время зависит от скорости чтения, которую ещё надо измерить.

Вопросы заказчику и допущения

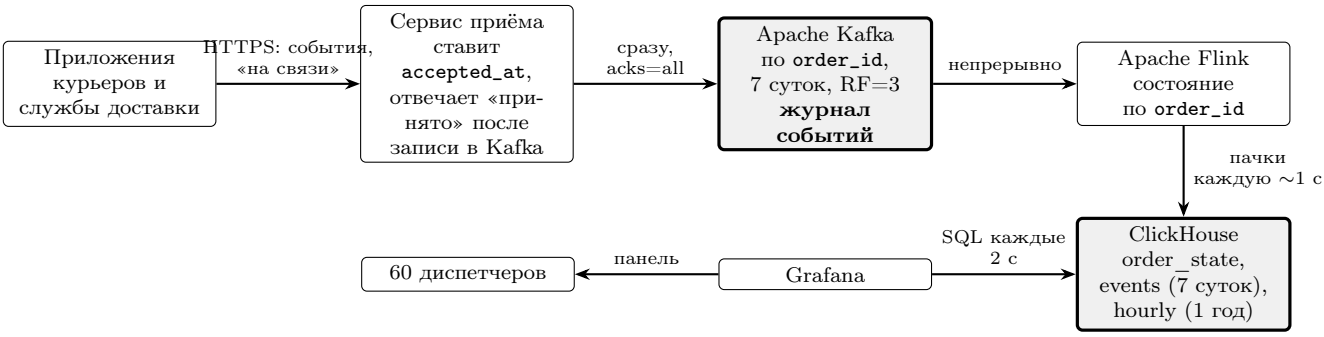
1. В каком виде приходят исправления? — Допущение: строки с разницей со знаком (сторно или доплата), у каждой уникальный `correction_id`.
2. Какова скорость канала от хранилища до сервера? — Допущение: не меньше 1 Гбит/с, перед запуском проверить замером.
3. Что показывать, если к 08:00 отчёт не готов? — Допущение: последний опубликованный отчёт с плашкой «Отчёт за D задерживается» + в 07:45 Airflow присылает команде оповещение.

Кейс 2. Контроль выполнения доставок

1. Требования

- Потоковая обработка: в среднем 10 000 событий/с, в пике 30 000, событие ∼500 байт. События приходят повторно, не по порядку или пачкой после 10 минут без связи.
- Результат: последний статус каждого заказа, давность сведений о нём, показатели опозданий и «зависших» заказов.
- 95% принятых изменений должны попасть в показатель на панели не позже чем через 5 с после приёма.
- Исходные события хранятся 7 суток для пересчёта, часовые итоги — год. Панель читают 60 диспетчеров.
- При сбое данные не теряются. Временная задержка допустима, но диспетчер должен её видеть.

2. Схема



3. Обоснование

Как событие становится состоянием и показателем. Сервис приёма ставит `accepted_at`, пишет событие в Kafka и только после подтверждения записи отвечает приложению «принято». Если ответ потерялся, приложение отправит событие ещё раз, дубль отсеется дальше. Kafka раскладывает события по `order_id`, поэтому события одного заказа попадают в одну партицию. Flink хранит для каждого заказа максимальную версию, статус, `status_changed_at` и `last_seen`. Примерно раз в секунду он пишет изменённые строки в `order_state` в ClickHouse. Таблица на движке ReplacingMergeTree, поэтому для каждого заказа остаётся строка с максимальной версией. Показатели панели — SQL-запросы Grafana к `order_state`. Граница проходит по записи в ClickHouse: до неё — обработка потока изменений, после — только чтение. Grafana и диспетчеры на поток не влияют.

Повторы, опоздания, очередь после сбоя. Событие с версией не больше сохранённой — дубль или запоздавший старый статус. Состояние заказа оно не меняет, попадает только в историю. Запись в ClickHouse идёт по ключу (`order_id`, версия), так что повторная запись ничего не портит. Часовые итоги считаются по времени события: опоздавшее событие Flink досчитывает в свой час, совсем поздние поправки пересчитываются из 7-суточной истории. После сбоя Flink поднимается из последней контрольной точки (состояние + позиции чтения в Kafka) и дочитывает журнал с того же места. При скорости обработки ∼60 000 событий/с накопившиеся 6 млн событий разбираются примерно за $6 \cdot 10^6/(60\,000 - 10\,000) \approx 120$ с. Пока система догоняет поток, на панели висит плашка «данные задерживаются на X с», задержка считается по отставанию чтения из Kafka.

«Не менялся» или «нет данных», отсчёт 5 с. Если `last_seen` свежий, а `status_changed_at` старый, статус просто не менялся: курьер на связи, но долго едет. Если `last_seen` старше порога, от курьера давно ничего не приходило, и на панели вместо «опаздывает» стоит серая отметка «нет свежих данных с HH:MM». Отдельный индикатор показывает отставание самой обработки, чтобы сбой системы не путали с молчанием курьеров. 5 с отсчитываются от `accepted_at`, то есть от момента приёма центральной системой. Время события на телефоне не подходит: оно может быть на 10 минут старше.

Расчёт объёма. 10 000 × 500 Б = 5 МБ/с, за сутки 5 МБ/с × 86 400 с = 432 ГБ. Пик — 15 МБ/с, за 7 суток ∼ 3 ТБ. Сверху добавятся: репликация Kafka с RF=3 (∼ 9,1 ТБ в сумме), копия событий в ClickHouse за 7 суток с двумя репликами (∼ 6 ТБ до сжатия, после сжатия в разы меньше), служебные данные (индексы и заголовки Kafka, позиции чтения, контрольные точки Flink, метаданные), запас свободного места на дисках 30–50%. Часовые итоги за год почти не занимают места.

Проверка задержки полного пути. Два способа. Метрика: ClickHouse записывает время вставки каждой строки, проверяем, чтобы $p95$(время вставки − `accepted_at`) $\leq$ 5 с. Нагрузочный тест: 30 000 событий/с + сценарий «10 минут без связи, затем пачка».

Альтернатива: приложения пишут напрямую в PostgreSQL. Отказался: база получала бы до 30 000 обновлений/с по одним и тем же строкам без буфера на пики и пачки после обрыва связи. Журнала для пересчёта за 7 суток нет, а любой сбой БД сразу превращается в ошибки у курьеров и потерянные события.

4. Ограничения

Ради свежести данных усложнил систему: три распределённые системы (Kafka, Flink, ClickHouse), которые надо обслуживать. Данные на панели согласуются с задержкой: панель отстаёт на несколько секунд, до 5% изменений появляются позже 5 с, во время сбоя требование временно нарушается. Данные при этом не теряются, задержку видно на панели. Grafana — только для просмотра. Если диспетчерам понадобится что-то делать с заказом (связаться с курьером, переназначить), можно сделать отдельное веб-приложение поверх того же ClickHouse.

Вопросы заказчику и допущения

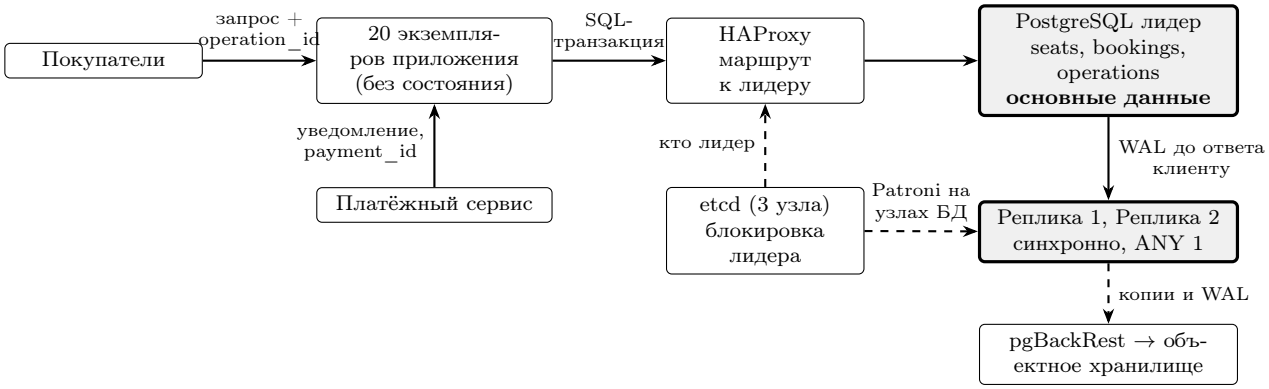
1. Отправляет ли приложение курьера регулярный сигнал «на связи»? — Допущение: да, раз в минуту.
2. Сколько времени без изменений считать подозрительным? — Допущение: у каждого статуса свой порог, которые лежат в справочнике и меняются без переразвёртывания.
3. По какому времени считать часовые итоги? — Допущение: по времени события, опоздавшее событие досчитывается в свой час.

Кейс 3. Система бронирования билетов

1. Требования

- Операционная (OLTP) нагрузка: до 3 000 коротких транзакций/с (резерв, подтверждение, отмена). Данных ∼80 ГБ, помещаются на один сервер БД.
- Одно место — один покупатель. Повторные запросы клиента и платёжного сервиса не должны создавать дублей.
- Подтверждённые брони не теряются при отказе сервера. Приём новых бронирований можно остановить максимум на 60 с.
- При разрыве сети корректность важнее доступности: лучше отказать части запросов.

2. Схема



3. Обоснование

Где принимается решение о резерве. Окончательное решение принимает PostgreSQL одной транзакцией:

```
UPDATE seats SET status='reserved', hold_id=:h, hold_until=now()+interval '10 min'
    WHERE event_id=:e AND seat_id=:s AND (status='free' OR hold_until<now())
```

Конфликт двух запросов A и B на последнее место. A блокирует строку места и меняет её, B ждёт снятия блокировки. Когда A фиксирует транзакцию, база заново проверяет условие для B, видит активный резерв и не обновляет ни одной строки. B получает «место занято». Все 20 экземпляров приложения работают с одной строкой на одном лидере, поэтому двойной резерв невозможен. Просроченные резервы освобождаются за счёт условия `hold_until<now()` + фоновое задание их подчищает.

Что означает подтверждение и что увидит клиент при повторе. Ответ «место зарезервировано до HH:MM» или «билет оплачен» уходит клиенту только после фиксации транзакции на лидере и хотя бы на одной реплике (`synchronous_commit=on`, `synchronous_standby_names='ANY 1 (r1, r2)'`). Клиент передаёт `operation_id`, в той же транзакции результат пишется в `operations` с первичным ключом `operation_id`. Если клиент после тайм-аута повторит запрос, база найдёт ключ и вернёт сохранённый ответ: тот же резерв с тем же сроком или тот же отказ. Второго резерва не будет, «место занято» клиент тоже не увидит. Оплата подтверждается запросом `UPDATE ... SET status='sold' WHERE hold_id=:h AND status='reserved'`, вместе с ним записывается `payment_id`. Если платёжный сервис пришлёт то же уведомление ещё раз, система увидит, что место уже продано по этому платежу, и ответит «успешно».

Сохранность и восстановление. Три узла PostgreSQL с синхронной репликацией ANY 1: при отказе любого одного узла подтверждённые данные не пропадают, запись не останавливается. Patroni на каждом узле + etcd (3 узла, консенсус) следят, чтобы лидер был ровно один. Если лидер упал, они переключают работу на реплику примерно за 30 с (время жизни блокировки лидера), это укладывается в минуту. HAProxy переводит трафик на нового лидера. Старый лидер, отрезанный от etcd, теряет блокировку и перестаёт принимать запись. Без подтверждения реплики он всё равно ничего не зафиксирует, поэтому двух лидеров одновременно не бывает. pgBackRest хранит резервные копии и WAL на случай логической ошибки. При ошибке приложение повторяет запрос с тем же `operation_id`, это безопасно.

Проверка при отказе. На стенде нагрузка 3 000 операций/с с гонками за последние места + проверка трёх сценариев: аварийная остановка лидера (`kill -9` или выключение), изоляция лидера от сети, отключение реплики. Проверка пройдена, если переключение занимает не больше минуты, каждая подтверждённая клиенту операция (по журналу генератора нагрузки) есть в БД, нет мест с двумя активными бронями, повтор с тем же `operation_id` возвращает тот же ответ.

Альтернатива: решение в памяти приложения. Отказался: 20 экземпляров не видят решений друг друга, два из них могут одновременно продать одно место. Если экземпляр упадёт, его резервы пропадут. Чтобы синхронизировать экземпляры, пришлось бы писать свой протокол согласования, то есть заново делать то, что уже есть в СУБД.

4. Ограничения

Пожертвовал доступностью ради корректности (CP). Во время переключения до минуты нельзя бронировать, при разрыве сети часть запросов получает отказ, синхронная репликация добавляет задержку к каждой записи. Вся запись идёт через одного лидера, поэтому расти можно только за счёт более мощного сервера. При большем росте нагрузки данные придётся разделить по мероприятиям.

Вопросы заказчику и допущения

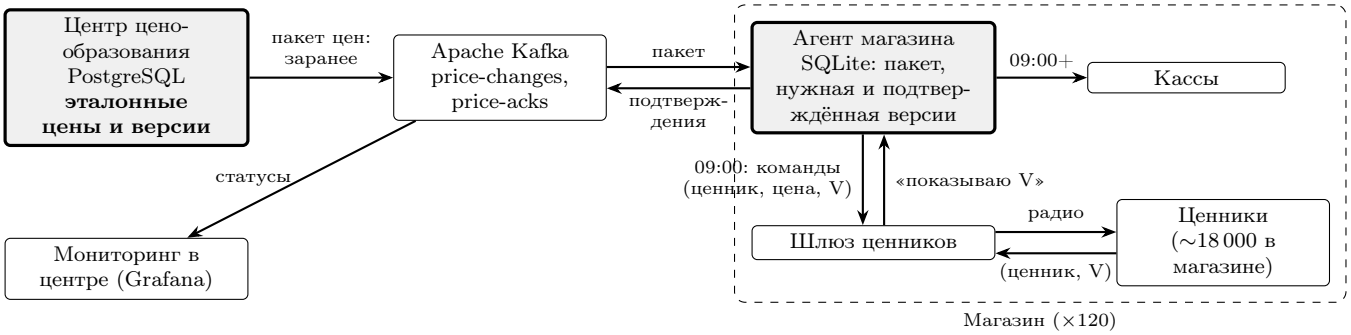
1. Что делать, если оплата пришла после истечения резерва? — Допущение: если место ещё свободно, покупка подтверждается. Если его занял другой покупатель, деньги автоматически возвращаются + клиент получает уведомление.
2. Сколько хранить ответ по `operation_id`? — Допущение: 24 часа, старые записи удаляет фоновое задание.

Кейс 4. Обновление электронных ценников

1. Требования

- Всего $120 \times 400 = 48\,000$ изменений (товар в конкретном магазине). Объём маленький, главная сложность — согласовать состояние в распределённой системе.
- Цена товара на кассе и на ценнике в одном магазине должна совпадать. Применение изменения подтверждает само устройство: ценник присылает свой идентификатор + версию цены.
- До 3% затронутых ценников (12 на магазин, до 1 440 по сети) недоступны до 20 минут. Связь с центром пропадает до 10 минут. Ценник не умеет принять цену заранее, кассу и ценник нельзя переключить одновременно. Команды и подтверждения повторяются и опаздывают.

2. Схема



3. Обоснование

Совместимы ли пожелания с условиями. Нет. Ценник меняет цену сразу после получения команды, поэтому отправлять команды можно не раньше 09:00, иначе акционная цена появится на полке раньше времени. До 3% ценников недоступны до 20 минут, а кассу и экран нельзя переключить одновременно. Значит, в 09:00 часть ценников гарантированно покажет старую цену. Предлагаю согласовать послабление: правило «касса не дороже ценника» + окно, за которое цены должны сойтись. Цена снижается: касса переключается ровно в 09:00, ценник догоняет позже. Пока он не обновился, покупатель платит меньше, чем написано на ценнике. Права покупателя не нарушены, но часть покупателей не увидит скидку на полке. Цена растёт: касса переключается только после того, как ценник подтвердил новую версию. До этого товар продаётся по старой, более низкой цене, магазин недополучает выручку. Касается это максимум 3% позиций и длится примерно до 20 минут.

Путь цены и подтверждения. Центр собирает для магазина пакет: ценник, товар, цена, версия V, направление изменения, время начала 09:00. Предварительный пакет уходит за час до закрытия магазина, финальный — при закрытии, круглосуточным магазинам — в 08:00. Каждая новая версия заменяет предыдущую. Kafka доставляет пакет агенту магазина по ключу `store_id`, после обрыва связи агент продолжает читать с того же места. Агент сохраняет пакет в SQLite и сообщает центру «пакет версии V получен». До 09:00 на кассы и ценники ничего не отправляется. В 09:00 агент переключает кассы для подешевевших товаров + отправляет команды шлюзу. Ответ шлюза «принято» значит только, что команда отправлена. Когда ценник подтверждает показ версии V, агент включает новую цену на кассе для подорожавших товаров. Статусы уходят в центр через Kafka. Цена опубликована, когда касса подтвердила применение V и ценник подтвердил показ V.

Работа без связи, недоступный ценник, старая команда. Всё нужное для 09:00 уже лежит в магазине, агент переключает цены сам, без центра. Подтверждения копятся в SQLite и уходят в центр после восстановления связи. У каждого сообщения ключ (магазин, ценник, версия), поэтому повторная доставка ничего не испортит. Пока связи нет, центр показывает «нет данных от магазина с HH:MM», а не «ошибка». Заранее центр следит, чтобы финальный пакет получили все магазины. Для каждого ценника агент хранит две версии: нужную и подтверждённую. Недоступному ценнику команда повторяется с растущим интервалом, касса держит старую цену подорожавшего товара. Если за 20 минут подтверждения нет, сотрудник получает задачу. Запоздавшая старая команда может перезаписать экран. Тогда ценник пришлёт подтверждение со старой версией, агент это заметит и отправит нужную версию снова. Пакеты из центра с версией ниже уже полученной агент отбрасывает.

Сценарий частичного обновления. Магазин №XX, 09:05. Новую цену подтвердили 388 ценников из 400, 12 молчат (села батарея, помехи). Пять из них — подорожавшие товары, касса пока продаёт их по старой цене. Ещё один ценник подтвердил V, но в 09:10 до него дошла задержавшаяся команда прошлой акции, и он подтвердил V−1. Обнаружение: агент сравнивает нужную и подтверждённую версии каждого ценника и повторно отправляет V. В центр уходит сводка «магазин XX: 388 из 400, 12 ждут дольше 5 минут». В 09:20 сотрудники получают список этих ценников с местами на полках.

Альтернатива: центр сам рассылает всё в 09:00. Отказался: если в 09:00 связь с магазином пропадёт на 10 минут, акция в нём не начнётся. Центр не знает, что на самом деле показывают экраны, магазины не могут работать самостоятельно.

4. Ограничения

Отказался от одновременного переключения ради правила «покупатель не переплачивает». Цены на кассе и ценнике расходятся примерно до 20 минут, но только у 3% ценников или меньше. Цена этого — недополученная выручка на подорожавших товарах + часть скидок не видна на полке. Плюс в каждом из 120 магазинов надо поддерживать агента и локальную базу.

Вопросы заказчику и допущения

1. Как и когда формируются цены? — Допущение: предварительные цены — за час до закрытия магазина, при закрытии их корректируют по продажам последнего часа. Круглосуточные магазины получают цены в 08:00.
2. Что делать с ценником, который молчит дольше 20 мин? — Допущение: сотрудник проверяет или меняет ценник либо вешает бумажный. Касса держит цену по правилу «не дороже ценника».
3. Какое окно расхождения допустимо? — Допущение: 97% совпадений к 09:05, 100% к 09:30.